

Project FINALBYTE (1.3)

Name: FINALBYTE

Subtitle: Unified audio-visual expansion system for 8-bit platforms (Atari 800XL / 130XE, Commodore 64, ZX Spectrum)

Project Goal

FINALBYTE is a cross-platform enhancement system for classic 8-bit computers. It adds modern audio, graphics, input and asset-processing capabilities while keeping the original 8-bit computer responsible for the game itself.

FINALBYTE is designed as an external **graphics / audio / I/O coprocessor**, not as a replacement computer hidden behind the host.

FINALBYTE provides:

-  Wavetable and sampled audio with effects.
-  A synchronized graphics layer with sprites, tiles, HUDs and full-screen generated scenes.
-  Modern optional USB input devices.
-  Sprite Kitchen and Sample Kitchen load-time asset generation.
-  Optional collision acceleration and other bounded helper services.
-  A unified logical command protocol across all supported hosts.

The host continues to own game state, rules, scripting, AI and other game logic unless a future extension explicitly defines an accelerator service.

Supported Platforms

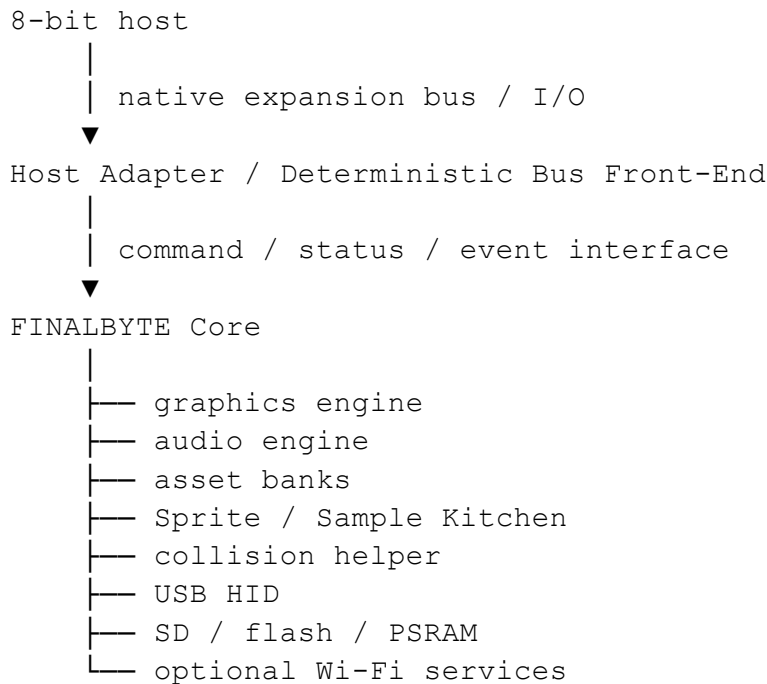
Initial target hosts:

- Atari 800XL / 130XE
- Commodore 64
- ZX Spectrum 128K / AY-compatible systems

FINALBYTE uses a common software API, but the physical transport and address decoding are adapted to each platform.

System Architecture

FINALBYTE is divided into three logical layers:



► FINALBYTE Core

Primary implementation target:

- **ESP32-S3** for the main programmable engine.

Alternative or specialized implementations may use:

- RP2040 / RP2350
- STM32
- CPLD
- FPGA

A small deterministic logic device or discrete latch/decode logic may be used between the legacy host bus and the main MCU.

The MCU is not required to respond directly to every legacy CPU bus cycle in software.

Core responsibilities

- Sound synthesis and sample playback.
 - Sprite, tile and HUD rendering.
 - FINALBYTE framebuffer generation.
 - Internal FINALBYTE-layer compositing.
 - Asset-bank management.
 - Kitchen recipe execution during loading.
 - USB HID input handling.
 - SD / flash storage access.
 - Optional network functionality.
-

Host Communication

1. Unified logical protocol

FINALBYTE 1.3 separates the **logical command protocol** from the **physical host transport**.

A game uses the same conceptual operations on every platform:

```
PLAY_SAMPLE
NOTE_ON
SET_SPRITE
MOVE_SPRITE
SET_TILE
SET_GFX_BANK
SET_SFX_BANK
READ_STATUS
READ_COLLISION
UPLOAD_DATA
...
```

The exact machine-code wrapper differs by host, but the command semantics remain common.

This permits portable game engines and shared source-level FINALBYTE libraries without forcing all platforms to emulate an identical physical memory map.

2. Host-specific transport

Commodore 64

Preferred transport:

- Expansion-port I/O space.
- Primary candidate: **IO2** / **\$DF00–\$DFFF**.

Typical implementation:

```
$DF00  COMMAND
$DF01  ARG0
$DF02  ARG1
$DF03  ARG2
$DF04  STATUS
$DF05  DATA
...
```

Exact register allocation is defined by the platform protocol document rather than this architecture overview.

Atari 800XL / 130XE

FINALBYTE uses an Atari-specific expansion interface rather than assuming the old provisional \$D700+ mapping.

Preferred implementation targets the machine's external expansion facilities, with the final register range and electrical interface defined by the Atari adapter specification.

Possible adapter implementations include PBI/ECI-oriented designs where appropriate.

ZX Spectrum

FINALBYTE uses Z80 I/O transactions through a reserved/decoded port scheme.

Typical software access:

```
OUT (port),A
IN A,(port)
```

The exact port decode and conflict-avoidance rules are defined by the ZX adapter specification.

3. Deterministic bus front-end

FINALBYTE 1.3 does not require the ESP32 firmware scheduler to meet every legacy bus-cycle deadline directly.

A host adapter may contain:

- address decoder,
- write latch,
- readable status registers,
- FIFO,
- level shifting,
- interrupt/event latch,
- small CPLD or equivalent deterministic logic.

Typical path:

```
Host CPU bus
    ↓
address / I/O decode
    ↓
latch / register / FIFO
    ↓
FINALBYTE MCU
```

This keeps timing-critical legacy-bus behavior deterministic while allowing the main MCU to concentrate on rendering, audio and higher-level services.

4. Command FIFO

Commands that do not require immediate register semantics may be queued in a small FIFO.

A command packet may contain:

```
opcode
length / flags
arguments
optional payload reference
```

The host can check **READY**, **BUSY**, **FIFO_FULL** and error state through status registers.

5. Read-back services

FINALBYTE may expose bounded read-back data such as:

- device/version identification,
- current bank state,
- collision events,
- input state,
- command completion,
- error flags,
- available FIFO space.

Large assets should normally reside on FINALBYTE-local SD / flash storage instead of being streamed continuously through the 8-bit host bus.

Passive Host Sniffing

Where electrically practical, a host adapter may observe selected writes without changing normal machine behavior.

Possible uses include:

- SID register activity on C64,
- POKEY activity on Atari,
- AY / beeper activity on ZX systems.

Passive sniffing is optional and platform-specific.

The original sound hardware remains functional; FINALBYTE may use observed activity to synchronize enhancement audio, visualization or effects.



Video Architecture

1. Design change in FINALBYTE 1.3

FINALBYTE 1.3 explicitly removes the requirement for true multi-level alpha blending against the original analog/composite host video.

FINALBYTE does **not** need to digitize every host pixel, decode the original composite signal, blend it numerically and re-encode it.

Instead, FINALBYTE generates its own synchronized video layer and selects between:

0 = original host video

1 = FINALBYTE-generated video

This is the **binary host-video compositor**.

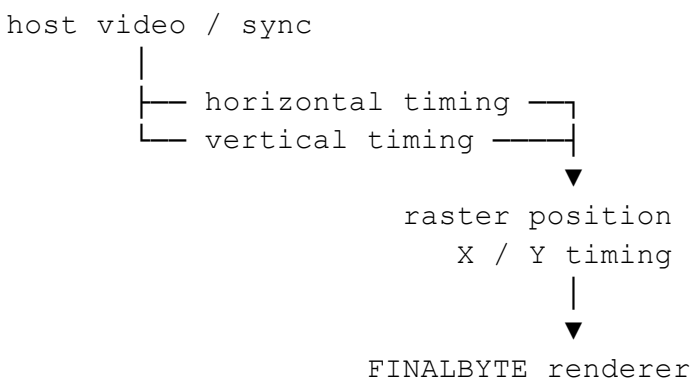
2. Horizontal and vertical synchronization

The FINALBYTE graphics layer must synchronize to both horizontal and vertical host timing.

Depending on the host adapter and available signal, this may use:

- HSYNC + VSYNC,
- composite sync,
- recovered horizontal / vertical timing,
- equivalent machine-specific timing signals.

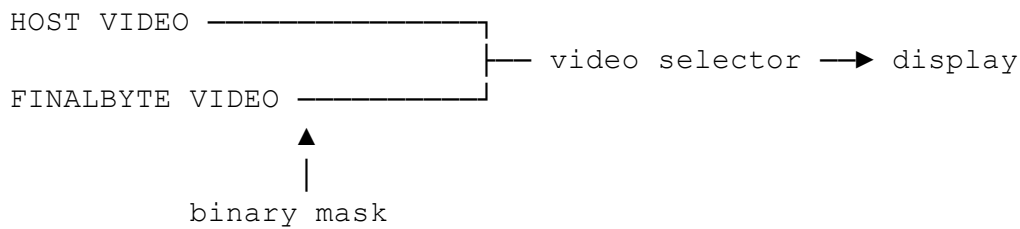
Conceptually:



Synchronizing only to VSYNC is not sufficient for pixel-positioned overlays.

3. Binary host / FINALBYTE selection

The final output stage performs a hard pixel/source selection:



For each rendered position:

- mask **0** passes the original host video,
- mask **1** selects the FINALBYTE-generated video.

No partial alpha value exists between the original host signal and FINALBYTE video in the base architecture.

4. Internal FINALBYTE alpha

Multi-level alpha is still allowed **inside the FINALBYTE-generated graphics layer**.

For example:

```
FINALBYTE background
+
FINALBYTE sprite alpha 0..15
+
FINALBYTE particle alpha 0..15
↓
FINALBYTE final pixel
↓
binary host/FINALBYTE selector
```

Therefore FINALBYTE may retain:

- 4-bit internal alpha,
- translucent FINALBYTE HUD elements,
- fades between FINALBYTE assets,
- soft particle effects,
- palette transparency,
- internal sprite compositing.

The restriction applies only to mixing FINALBYTE pixels with the original analog host pixels.

5. Video modes

Overlay Off

original host video only

FINALBYTE graphics are disabled; audio and other services may remain active.

Overlay HUD

FINALBYTE replaces only selected pixels/regions while the remaining screen passes through from the host.

Typical uses:

- HUD,
- mouse cursor,
- labels,
- indicators,
- enhanced sprites,
- particles,
- menus.

Overlay Full

FINALBYTE selects its generated layer for the complete active area.

Typical uses:

- cutscenes,
- maps,
- adventure screens,
- high-detail menus,
- hardware-accelerated renderers,
- full FINALBYTE tile/sprite scenes.

The original host remains responsible for game logic even when its native video is temporarily fully replaced.

6. Host-specific video adapters

Video timing and analog electrical details differ between the supported machines.

FINALBYTE therefore permits host-specific video adapters that handle:

- signal levels,
- synchronization extraction,
- host-video passthrough,
- switching/multiplexing,
- machine-specific video connectors.

The FINALBYTE core consumes a normalized timing interface wherever practical.



FINALBYTE Graphics Engine

Target logical canvas:

- **320×200** reference resolution.
- 16- or 32-color indexed palette modes.
- Internal transparency / optional 4-bit alpha within FINALBYTE graphics.

The precise generated electrical video format may vary by hardware adapter.

Sprite support

- Positioning.
- Z-order.
- Animation.
- Palette selection.
- Binary transparency.
- Optional internal alpha.
- Origin / hotspot metadata.
- Collision metadata.
- HUD elements, icons and labels.

Tile system

Default lightweight profile:

- tile size: 16×16 pixels,
- default visible grid: approximately 20×14 tiles depending on active display geometry,
- multiple logical layers,
- smooth or stepped scrolling according to renderer mode,
- HUD layer independent from the game field.

Larger maps may be stored on FINALBYTE storage and windowed into active memory.

Sprite and Sample Kitchen

FINALBYTE 1.3 incorporates the separate **Sprite and Samples Kitchen 1.3** specification.

The Kitchen is a load-time asset compiler. It creates derived assets during game initialization or bank loading so that the 8-bit host does not pay the processing cost during gameplay.

Sprite Kitchen includes

- Shift.
- Mirror X/Y.
- Rotate 90° / 180° / 270°.
- Scale 50% / 200%.
- Crop.
- Slice.
- Trim.
- Repeat X/Y.
- Palette swap.
- Color key.
- Mask / mask merge.
- Composition.
- Origin / hotspot / collision metadata.

Load-time blend modes

Sprite Kitchen may combine two images, or an image with a constant color plane, using inexpensive deterministic operations:

- NORMAL / COPY,
- MULTIPLY,
- SCREEN,
- OVERLAY,
- ADD,
- SUBTRACT,
- MIN,
- MAX,
- XOR.

Constant planes include:

BLACK
WHITE
RED
GREEN
BLUE
CYAN
MAGENTA
YELLOW

A load-time blend may optionally use strength 0–15.

This does **not** imply multi-level alpha against the original host video. The operation produces a finished derived asset before normal runtime rendering.

Sample Kitchen includes

- Crop.
 - Reverse.
 - Volume adjustment.
 - Speed/pitch-by-resampling.
 - Mix.
 - Fade in/out.
 - Normalize.
 - Silence insertion.
 - Append.
 - Loop-region metadata.
-

Sound Engine

FINALBYTE provides an external sound engine independent from the limited native host audio hardware.

Core capabilities:

- 8/12/16-bit sample sources where supported by implementation.
- Wavetable playback.
- Velocity / volume.
- Pitch control.
- Playback offset.
- Looping.
- Stereo output.
- Multiple simultaneous voices according to implementation limits.

Optional effects:

- reverb,
- delay / echo,
- filtering,
- saturation,
- pitch manipulation.

Representative commands:

```
NOTE_ON
NOTE_OFF
PLAY_SAMPLE
STOP_SAMPLE
SET_VOLUME
SET_PAN
SET_PITCH
FX_ON
FX_OFF
```



Asset Banks

Sound Sample Banks

Four logical banks are defined in the default profile:

- **BANK 0** — built-in / core sound assets.
- **BANK 1–3** — game-defined or scene-defined banks.

Example API:

```
FINALBYTE_SET_SFX_BANK(n);          // 0..3  
FINALBYTE_PLAY_SAMPLE(0x12);
```

Sprite / Tile Graphics Banks

Four logical graphics banks are defined in the default profile:

- **BANK 0** — common/core assets.
- **BANK 1–3** — game, level, scene or theme banks.

Example API:

```
FINALBYTE_SET_GFX_BANK(n);          // 0..3
```

Banks may contain:

- sprites,
- animation frames,
- tiles,
- palettes,
- masks,
- HUD elements,
- metadata,
- Kitchen source and/or generated assets.

Bank counts are a default software profile, not necessarily a hard physical memory limit.



USB Keyboard and Mouse

On implementations with USB-host capability, especially ESP32-S3-based cores, FINALBYTE may provide direct USB HID input.

Supported use cases include:

- mouse movement,
- mouse buttons,
- keyboard scanning,
- GUI control,
- adventure games,
- strategy / RTS games,
- independent FINALBYTE cursor rendering.

The mouse cursor may be rendered directly by FINALBYTE while host software reads logical cursor/button state through the host interface.



Collision Acceleration

FINALBYTE may perform sprite collision checks in the external graphics system.

The host remains responsible for deciding what a collision means in game logic.

Examples:

```
sprite overlap → FINALBYTE detects
                → host reads event
                → host applies damage / pickup / trigger
```

Collision event FIFO / buffer

Recommended representation:

```
SPRITE_ID_A
SPRITE_ID_B
FLAGS
...
```

The exact memory or I/O mapping is host-specific.

Optional pair-query representation

Implementations may also expose a compact pair/bitmask query mechanism where useful.

Collision acceleration is a helper service and must not silently own game-state rules.



Timing, Refresh and Game Simulation

FINALBYTE graphics refresh and 8-bit host simulation are intentionally decoupled.

A game may update its logical world less frequently than FINALBYTE refreshes the screen.

Example:

```
Host simulation:      8-15 Hz
FINALBYTE animation: 24-30 fps
Video scan/output:    synchronized to host display timing
```

FINALBYTE may interpolate or continue previously submitted animation state without forcing the host CPU to submit a complete new frame every display refresh.

This allows visually smooth games while keeping game logic on the original CPU.

Optional cinematic modes may use film-like animation rates such as 24 fps.



Wi-Fi Connectivity

ESP32-based implementations may provide optional Wi-Fi services.

Possible uses:

- asset / modpack download,
- software/version verification,
- bank updates,
- optional multiplayer handshake,
- development/debug services.

Configuration may be loaded from SD storage or provided through a FINALBYTE overlay menu.

Example package contents:

```
gfx.bank
sfx.bank
manifest.yaml
```

Network access is an enhancement and is not required for normal offline operation.



Responsibility Boundary

FINALBYTE follows a simple rule:

The host runs the game. FINALBYTE accelerates presentation and bounded peripheral-style services.

Normally owned by the 8-bit host

- game state,
- rules,
- mission logic,
- scripting,
- inventory,
- economy,
- AI,
- combat decisions,
- save-game semantics.

Normally owned or accelerated by FINALBYTE

- sprite/tile rendering,
- graphical composition,
- animation playback,
- audio playback and effects,
- modern HID input,
- asset loading and transformation,
- collision detection/query,
- raster-synchronized overlay generation.

Future accelerator extensions may add services such as specialized geometry or rendering helpers, but they should remain explicit APIs rather than silently moving the whole game into FINALBYTE.

Compatibility Philosophy

- No permanent modification of the original computer is required.
- Host adapters should use expansion facilities and conflict-safe I/O decoding appropriate to each machine.
- Games may detect FINALBYTE through a platform-specific handshake exposed through the common API.
- A game may provide a stock-machine or "Lite" path when FINALBYTE is absent.
- FINALBYTE-specific enhancements should degrade cleanly where practical.
- The physical interface is platform-specific; the logical programming model is shared.

Example detection flow:

```
host starts
  ↓
platform FINALBYTE probe
  ↓
valid signature/version?
├─ yes → enhanced mode
└─ no  → stock/Lite mode
```



Example Use Cases

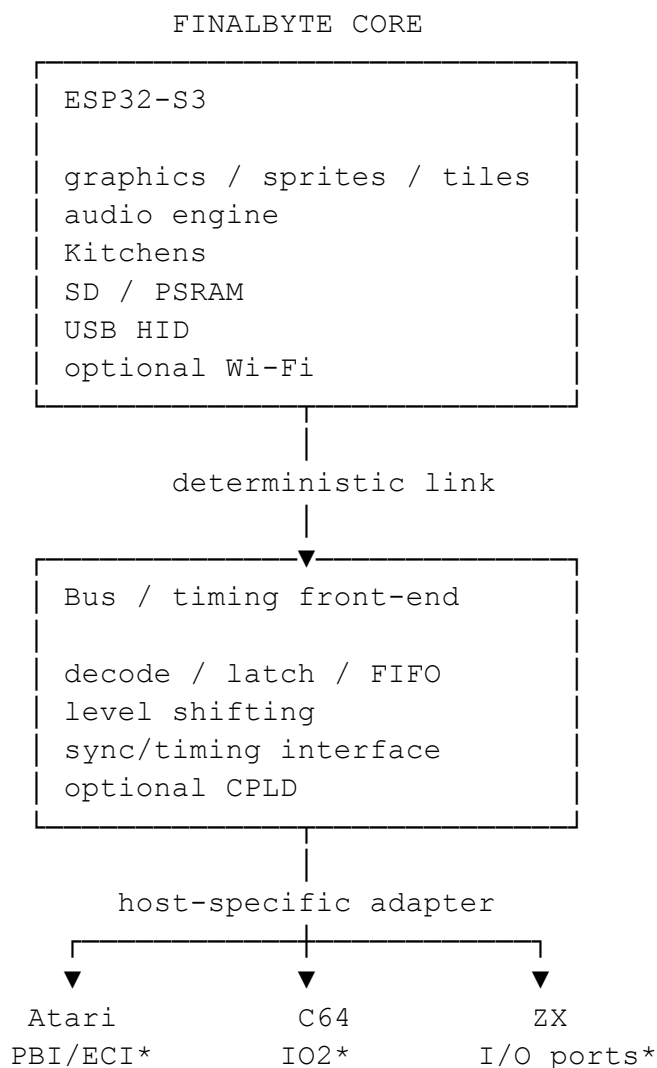
FINALBYTE can support:

- enhanced editions of classic-style platformers and action adventures,
 - cinematic games in the style of Another World / Flashback,
 - hardware-assisted raycasting / 3D presentation while host logic remains on the 8-bit CPU,
 - RTS / strategy games where the host runs simulation while FINALBYTE handles tiles, sprites, mouse and audio,
 - high-quality speech and sampled effects,
 - graphical adventure interfaces,
 - synchronized demo-scene visuals,
 - VJ/live systems using a retro computer as controller and logic host.
-



Recommended Hardware Partition

A practical FINALBYTE 1.3 implementation may use:



* Exact electrical and address definitions belong to the individual platform adapter specifications.

This separation keeps the legacy-facing side deterministic while allowing a modern MCU to perform the expensive graphics, audio and asset work.



FINALBYTE 1.3 Key Decisions

1. **Unified protocol, host-specific transport.**
The API is shared; physical registers/ports are platform-dependent.
 2. **Deterministic bus front-end.**
Timing-critical legacy bus transactions need not depend directly on MCU interrupt latency.
 3. **Horizontal + vertical video synchronization.**
FINALBYTE tracks complete raster timing, not VSYNC alone.
 4. **Binary host-video compositing.**
Original host video vs FINALBYTE video is selected with a 1-bit mask.
 5. **No true alpha blend against original composite video.**
FINALBYTE does not require full host-video digitization and re-encoding.
 6. **Internal FINALBYTE 4-bit alpha remains available.**
Multi-level alpha may be used while compositing FINALBYTE-generated assets and pixels.
 7. **Sprite / Sample Kitchen integrated as load-time asset compiler.**
Expensive transformations happen outside the 8-bit host gameplay loop.
 8. **Host remains the game computer.**
FINALBYTE is a graphics/audio/input coprocessor plus explicit bounded accelerators.
-



Status

- Architecture revision: **1.3**.
 - High-level host/graphics responsibility model defined.
 - Binary video compositor selected for the base design.
 - Sprite and Samples Kitchen 1.3 defined.
 - Platform-specific bus/register specifications still to be frozen.
 - Exact video adapter electrical designs still to be prototyped per host.
 - Planned open-source release: hardware + firmware + host libraries / SDK.
-

FINALBYTE = One spirit, three legends.

Together, we bring 8-bit creativity into a new golden age.